Ch. 5 Cheat Sheet
Hash Tables

The big idea

A hash table maps **key** $\to$ **bucket index** via a hash function h. Insert/find/remove become array-index operations. Expected cost: $O(1)$. Worst case: $O(n)$.

Hash function desiderata

1. **Deterministic.** Same key $\to$ same index.
2. **Fast.** Ideally a handful of integer ops.
3. **Uniform.** Keys spread across all buckets.

Classic methods: **division** ($k \bmod m$, pick m prime), **multiplication** ($\lfloor m(kA \bmod 1) \rfloor$ for $A \approx 0.618$), **mid-square** (square, take middle digits).

Load factor

$$\alpha = \frac{n}{\text{capacity}}$$

Chaining: works up to $\alpha \approx 1$; cost grows with α.
Open addressing: must keep $\alpha < 1$; performance degrades sharply as $\alpha \to 1$. Rule of thumb: resize at $\alpha = 0.75$.

Chaining

Each bucket = linked list. Collisions chain onto it.
Insert: hash $\to$ bucket $\to$ prepend node. $O(1)$.
Find: hash $\to$ bucket $\to$ walk chain. $O(1 + \alpha)$ expected.
Remove: same as find + unlink.
Deletion: easy – just remove from list.

Linear probing

On collision, try index $+1, +2, \ldots$ until empty.
Pros: great cache behavior (contiguous probes).
Cons: **primary clustering** – once a run starts, it grows and attracts more collisions.
Cost (expected, uniform hashing): insert $\approx \frac{1}{2}(1 + \frac{1}{1-\alpha})$; find-miss $\approx \frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$.

Quadratic probing

Try index $+1^2, +2^2, +3^2, \ldots$.
Pros: avoids primary clustering.
Cons: **secondary clustering** – keys with the same initial hash probe the same sequence. Needs capacity chosen so the probe sequence covers all slots (e.g., capacity = prime, $\alpha < 0.5$).

Double hashing

Two hash functions: $h_1(k)$ for initial slot, $h_2(k)$ for step size. Probe $h_1(k), h_1(k) + h_2(k), h_1(k) + 2h_2(k), \ldots$.
Pros: breaks both clustering modes; closest to uniform hashing in practice.
Cons: two hashes computed. h_2 must never return 0 and should be coprime with capacity.

Probe-table deletion: tombstones

Open addressing can't just erase a slot – that would break the probe chain of some later key. Instead, mark the slot as a **tombstone** ("was occupied, now empty"). Find skips past; insert may reuse. Periodic full rehash is needed to clean them up.

Resizing (rehashing)

Trigger: α exceeds a threshold (e.g., 0.75).
New capacity: usually $\approx 2\times$ old; pick the next prime if method needs it.
Every element must be rehashed – new indices come from new modulus. Simple copy is wrong.
Cost: $O(n)$ single rehash; amortized $O(1)$ per insert over sequence.

Direct hashing (perfect hashing)

If keys *are* small integers in a dense known range, use them as indices directly. No hash function, no collisions.
Use when: counting occurrences of ASCII chars (256-slot table), counting digit frequencies, histogramming ages, ...
Anti-use: sparse key space (10 employee IDs drawn from 9-digit SSNs $\rightarrow 10^9$-slot table).

Comparison at a glance

Strategy	Insert	Find	Notes
Chaining	$O(1)$	$O(1+\alpha)$	tolerates $\alpha \geq 1$
Linear probing	$O(1)$ avg	$O(1)$ avg	primary cluster
Quadratic probing	$O(1)$ avg	$O(1)$ avg	secondary cluster
Double hashing	$O(1)$ avg	$O(1)$ avg	closest to ideal

Hash table vs. BST vs. sorted array

	HT	BST	sorted arr
find	$O(1)^*$	$O(\log n)$	$O(\log n)$
insert	$O(1)^*$	$O(\log n)$	$O(n)$
min/max	$O(n)$	$O(\log n)$	$O(1)$
sorted iter	$O(n \log n)$	$O(n)$	$O(n)$
range query	bad	good	good
memory overhead	pointers + empty	2 ptrs/node	none

* amortized / expected.

Crypto hashing (5.9) is a different beast

Requirements: preimage resistance, second-preimage resistance, collision resistance, *slowness* (for passwords). MD5 and SHA-1 are broken for collision resistance; use SHA-256+. For passwords use bcrypt / scrypt / Argon2. Never roll your own.

Top gotchas

- Quoting hash lookup as $O(1)$ without saying *expected*.
- Open addressing deletion by clearing the slot (breaks chains).
- Poor hash function + adversarial input = hash flooding DoS.
- Resizing that copies but doesn't rehash.
- Using `std::unordered_map` when you need ordered iteration or range queries (use `std::map`).